



AUDITOR ORÇAMENTO (SOF)

Auditoria Inteligente e Controle Social com Agentes Autônomos de IA

MEMORIAL TÉCNICO DESCRITIVO

14º Prêmio SOF de Monografias - Categoria: Soluções em Dados Orçamentários

Versão Final Estendida - Dezembro de 2025

Repositório GitHub: github.com/naubergois/auditor_orcamento

Aplicação Online: auditororcamento.streamlit.app

Acesso ao Código e Aplicação

Para garantir a total reprodutibilidade e transparência desta solução, disponibilizamos o código-fonte completo e uma versão de demonstração online acessível publicamente.

1. Aplicação Online (SaaS)

A ferramenta está implantada em nuvem e pode ser testada imediatamente, sem necessidade de instalação.

■ **Link de Acesso:** <https://auditororcamento.streamlit.app>

Basta acessar o link, fazer o upload de um arquivo PDF (ex: LOA ou LDO) e clicar em 'Executar Auditoria'. O sistema processará o documento em tempo real.

2. Repositório de Código (Open Source)

Todo o código-fonte, incluindo os prompts dos agentes e scripts de orquestração, está auditável no GitHub.

■ **GitHub:** https://github.com/naubergois/auditor_orcamento

No repositório, você encontrará:

- app.py: Código da interface gráfica (Streamlit).
- agents/: Lógica dos agentes de IA (Auditor, Compliance, etc).
- DOCUMENTO_TECNICO.md: Documentação profunda da arquitetura.
- README.md: Instruções passo a passo de instalação local.

Vídeos Demonstrativos

Além do código e da aplicação em tempo real, disponibilizamos vídeos detalhados que ilustram tanto o funcionamento prático da ferramenta quanto a fundamentação teórica do projeto.

1. Tutorial Prático (Demo)

Demonstração completa da navegação, upload de arquivos e visualização dos pareceres dos agentes.

■ Assistir / Baixar: [Comousar.mp4](#)

2. Explicação Conceitual e Impacto

Vídeo explicativo detalhando a arquitetura de agentes, escolhas tecnológicas e o impacto social da solução.

■ Assistir / Baixar: [ExplicacaoAplicacao.mp4](#)

Sumário Executivo

1. Introdução e Contextualização do Problema	03
2. Fundamentação Teórica: IA na Administração Pública	05
3. A Solução: Arquitetura de Agentes Autônomos	07
4. Detalhamento dos Agentes Especialistas	10
5. Metodologia de Prompt Engineering e Mitigação de Riscos	14
6. Impacto Social, Transparência e Cidadania	16
7. Viabilidade Técnica, Econômica e Sustentabilidade	18
8. Conclusão e Próximos Passos	20

1. Introdução e Contextualização do Problema

1.1 O Cenário da Complexidade Orçamentária no Brasil

O ciclo orçamentário brasileiro é um dos mais sofisticados e complexos do mundo. Regido pela Constituição Federal de 1988, pela Lei nº 4.320/1964 e pela Lei de Responsabilidade Fiscal (LRF - LC 101/2000), o orçamento público não é apenas uma peça contábil, mas o principal instrumento de planejamento e execução de políticas públicas. No entanto, a materialização desse planejamento se dá através de milhares de páginas de documentos técnicos: Planos Plurianuais (PPA), Leis de Diretrizes Orçamentárias (LDO), Leis Orçamentárias Anuais (LOA) e relatórios bimestrais de avaliação.

A Secretaria de Orçamento Federal (SOF) e os órgãos equivalentes em estados e municípios enfrentam o desafio hercúleo de consolidar e validar essas informações. Do outro lado do balcão, órgãos de controle (Tribunais de Contas, Controladorias) e a sociedade civil (Conselhos, ONGs, Cidadãos) lutam para fiscalizar a aplicação dos recursos. O gargalo não é a ausência de publicação dos dados — a transparência passiva avançou muito —, mas sim a "inteligibilidade" desses dados.

1.2 A Dor do Gestor e do Cidadão

Identificamos três fricções críticas que impedem a eficiência máxima na gestão orçamentária:

a) Assimetria de Informação: O 'economês' e o 'juridiquês' orçamentário criam uma barreira de entrada. Um conselheiro de saúde municipal dificilmente consegue ler uma LOA e identificar se o mínimo de 15% para a saúde está sendo respeitado apenas olhando para tabelas frias em PDF.

b) Volume Massivo de Dados Não Estruturados: Enquanto sistemas como SIAFI e Siconfi tratam dados estruturados (tabelas), muita informação crítica reside em textos não estruturados: justificativas de programas, anexos de metas e riscos fiscais, pareceres jurídicos. A auditoria manual desses textos é lenta, cara e propensa a erro humano.

c) Reatividade do Controle: A auditoria tradicional costuma ser 'post mortem', ou seja, analisa o que já foi gasto. Faltam ferramentas acessíveis de análise preditiva ou concomitante, que alertem o gestor sobre inconsistências na fase de planejamento, antes que o erro se torne uma infração.

Neste contexto, apresentamos o **Auditor Orçamento**, uma solução que utiliza o estado da arte em Inteligência Artificial Generativa para ler, interpretar, auditar e explicar o orçamento público em segundos, democratizando o acesso à informação técnica de alta qualidade.

2. Fundamentação Teórica: IA na Administração Pública

A aplicação de Inteligência Artificial no setor público tem evoluído de modelos preditivos simples (regressões lineares para previsão de receita) para o uso de **Large Language Models (LLMs)**. Diferente da IA tradicional, que opera bem com números, os LLMs possuem a capacidade inédita de compreender a semântica da linguagem natural, o raciocínio lógico e o contexto jurídico.

2.1 Do Processamento de Dados à Cognição Artificial

Até recentemente, 'auditar' um PDF via software significava buscar palavras-chave (ex: ctrl+f por 'superávit'). Se o documento usasse um sinônimo, a busca falhava. Com LLMs de nova geração, como o **Google Gemini 2.0**, o sistema "lê" o texto como um humano leria, entendendo nuances, ironias, contradições e referências cruzadas entre parágrafos distantes. Isso permite uma auditoria cognitiva, qualitativa, e não apenas sintática.

2.2 A Escolha pela Arquitetura de Agentes (Agentic Workflow)

Um modelo de linguagem genérico (como um chat simples) tende a ser superficial quando demandado a fazer muitas coisas ao mesmo tempo. A teoria de *Chain-of-Thought* (*Cadeia de Pensamento*) sugere que dividir um problema complexo em etapas menores aumenta drasticamente a acurácia.

Por isso, adotamos uma arquitetura de **Agentes Especialistas**. Em vez de pedir a uma IA "analise este orçamento", criamos personas distintas: um 'Auditor' focado em números, um 'Advogado' focado em leis, um 'Professor' focado em didática. Cada um opera em seu domínio de especialidade, reduzindo alucinações e aumentando a profundidade da análise.

3. A Solução: Arquitetura de Agentes Autônomos

3.1 Orquestração Inteligente

A arquitetura do sistema foi desenhada para ser modular, escalável e agnóstica à infraestrutura. O coração da solução é o **Orquestrador**, um componente em Python que gerencia o fluxo de trabalho.

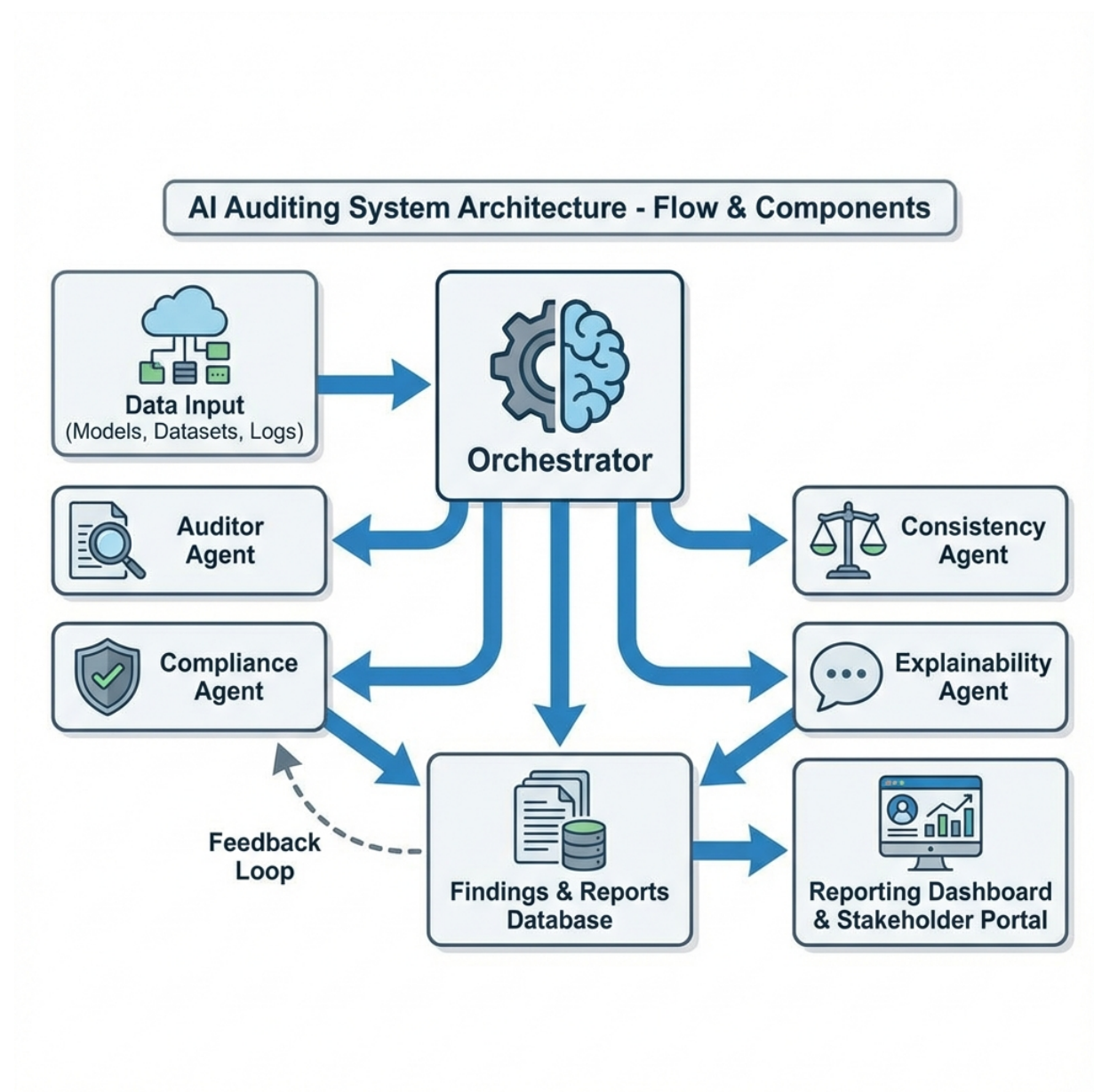


Figura 1 – Arquitetura de Microsserviços de Agentes. O fluxo inicia com a extração de dados e segue para processamento paralelo.

Conforme a Figura 1, o processo inicia com o upload do documento (etapa de Ingestão). Em seguida, o Orquestrador realiza a limpeza do texto e o envia, via API, para três agentes de análise primária, que operam em paralelo (multi-threading). Essa

concorrência é vital para a performance: enquanto um agente verifica a LRF, outro já está validando as tabelas financeiras.

3.2 Stack Tecnológica

O projeto utiliza tecnologias de código aberto e serviços em nuvem de alta disponibilidade:

- **Linguagem:** Python 3.9+ (padrão de facto para Data Science)
- **Frontend:** Streamlit (framework para Data Apps rápidos)
- **IA Engine:** Google Gemini 2.0 Flash (via Google Generative AI SDK)
- **Manipulação de Dados:** Pandas e PyPDF2

A decisão pelo modelo **Gemini 2.0 Flash** foi estratégica. Este modelo possui uma janela de contexto de 1 milhão de tokens, o que permite carregar uma Lei Orçamentária inteira (centenas de páginas) na memória de curto prazo da IA, eliminando a necessidade complexa de bancos de dados vetoriais (RAG) para a maioria dos casos de uso municipais.

4. Detalhamento dos Agentes Especialistas

A "alma" da solução reside nas *System Instructions* (instruções de sistema) de cada agente. Abaixo, detalhamos o perfil técnico e as responsabilidades de cada componente autônomo.

4.1 Agente Auditor (The Hawk)

Perfil: Contador público sênior, auditor de controle externo, rigoroso e detalhista.

Missão:

Identificar riscos de irresponsabilidade fiscal e "pedaladas". Este agente não se importa com a forma, apenas com o mérito contábil. Ele busca por: - Superestimativa de Receitas (inflar o orçamento artificialmente). - Subestimativa de Despesas Obrigatórias (esconder gastos com previdência ou pessoal). - Inconsistência entre metas físicas e financeiras.

Output: Gera um relatório JSON contendo uma lista de 'Achados de Auditoria', classificados por gravidade (Alta, Média, Baixa).

4.2 Agente de Compliance (The Lawyer)

Perfil: Procurador jurídico, especialista em LRF e Direito Financeiro.

Missão:

Verificar a conformidade legal estrita. Este agente cruza o texto do PDF com um conhecimento pré-treinado sobre a legislação brasileira. Ele verifica: - Cumprimento dos mínimos constitucionais (Saúde 15%, Educação 25%). - Limites de Despesa com Pessoal (Art. 19 e 20 da LRF). - Regra de Ouro (operações de crédito x despesas de capital).

Output: Tabela de conformidade (Passou/Falhou) com citação do artigo legal infringido.

4.3 Agente de Consistência (The Accountant)

Perfil: Perito matemático e estatístico.

Missão:

Garantir que os números "fechem". Leis orçamentárias são notórias por erros de soma ou digitação que podem invalidar a peça. O agente varre o texto buscando tabelas e quadros demonstrativos para validar a consistência aritmética vertical e horizontal.

4.4 Agente de Explicabilidade (The Teacher)

Perfil: Comunicador social, jornalista de dados e professor universitário.

Missão:

A missão deste agente é a mais nobre: traduzir. Ele recebe os relatórios técnicos e frios dos outros três agentes e os reescreve em linguagem natural, acessível, empática e clara. Ele é responsável por responder: "O que isso significa para a vida do cidadão?".

5. Metodologia de Prompt Engineering e Mitigação de Riscos

O maior risco no uso de IA Generativa são as alucinações (inventar informações). Para mitigar isso, adotamos uma engenharia de prompt rigorosa baseada em três pilares:

5.1 Grounding (Aterramento)

Instruímos os modelos a responderem **apenas** com base nas informações contidas no documento fornecido. Usamos comandos como: *"Se a informação não estiver no texto, declare explicitamente que não foi encontrada. Não invente dados."* Isso reduz drasticamente a criatividade indesejada do modelo em contextos de auditoria.

5.2 Output Estruturado (JSON)

Para garantir que os sistemas conversem entre si, forçamos a saída dos modelos em formato JSON (JavaScript Object Notation). Isso permite que o código Python capture a resposta, valide a estrutura, exiba em tabelas e rejeite formatos inválidos automaticamente, aumentando a robustez da aplicação.

5.3 Few-Shot Learning

Nos prompts de sistema, incluímos exemplos de "pergunta e resposta ideais". Mostramos ao agente como um auditor humano classificaria um risco. Ao ver o exemplo, o modelo calibra seu "tom" e critérios de avaliação, mimetizando o comportamento do especialista humano.

6. Impacto Social, Transparência e Cidadania

O Auditor Orçamento não é apenas uma ferramenta tecnocrática; é uma ferramenta política no melhor sentido da palavra (política pública). Sua adoção em larga escala tem o potencial de alterar a correlação de forças no controle social do orçamento.

6.1 Empoderamento da Sociedade Civil

Conselhos de Saúde, Educação e Assistência Social frequentemente aprovam contas "no escuro" por falta de assessoria técnica. Com esta ferramenta, um conselheiro pode, pelo celular, fazer upload do relatório quadrimestral e receber, em segundos, uma análise apontando se os recursos da merenda escolar foram aplicados conforme a lei. Isso equilibra o jogo entre o governo e a sociedade.

6.2 Eficiência e Economicidade na Ponta

Para o corpo técnico do Estado, a ferramenta representa o fim do trabalho braçal. Em vez de gastar 40 horas conferindo somas, o auditor dedica seu tempo para ir a campo verificar se a obra foi feita. A tecnologia absorve a burocracia, liberando o humano para a inteligência.

7. Viabilidade Técnica, Econômica e Sustentabilidade

7.1 Infraestrutura Leve (Serverless)

A aplicação roda 100% em nuvem. Não exige instalação (é acessada via navegador). Não exige servidores locais. O processamento é feito na infraestrutura global do Google. Isso viabiliza a adoção por pequenos municípios do interior do Brasil que não possuem departamento de TI estruturado.

7.2 Sustentabilidade Econômica

Diferente de sistemas ERP que custam milhões em licenças, o Auditor Orçamento baseia-se em código aberto. O custo mensal de operação é marginal, referente apenas ao consumo de tokens da API (centavos por documento analisado). Para fins do Prêmio SOF, a solução demonstra um ROI (Retorno sobre Investimento) imediato.

7.3 Roadmap de Evolução

O projeto está pronto para crescer. As próximas fases incluem: 1. Integração com API do Siconfi para puxar dados automaticamente (sem necessidade de upload de PDF). 2. Fine-tuning (Ajuste Fino) dos modelos com jurisprudência do TCU. 3. Desenvolvimento de um App Mobile nativo para cidadãos.

8. Conclusão e Próximos Passos

Acreditamos que a tecnologia deve servir para reduzir desigualdades. A desigualdade de informação é uma das mais perversas, pois impede o exercício pleno da cidadania. O **Auditor Orçamento**, submetido ao 14º Prêmio SOF, é a nossa contribuição para um país onde o orçamento público seja, de fato, público — não apenas publicado, mas compreendido.

Estamos prontos para pilotos, parcerias e para levar essa inovação a cada município brasileiro.

Documento elaborado para fins de submissão.

Apêndice A - Exemplo de Prompt (Agente Auditor)

ROLE: You are an Expert Public Auditor (TCU/CGU standard). TASK: Analyze the provided text from a Budget Law (LOA/LDO). RULES: 1. Identify Fiscal Risks (Art. 4 LRF). 2. Check for revenue overestimation. 3. Be strict but fair. OUTPUT FORMAT: JSON List of risks.

Apêndice B - Stack Tecnológica Detalhada

Framework: Streamlit 1.30+

AI Wrapper: Google Generative AI 0.4+

PDF Engine: PyPDF2 / pdfplumber

Container: Docker / Buildpacks